

CMPU4003 Advanced Databases

Week 1 Lab: Campus Events

From Databases 2 to Advanced Databases

Introductory lab.

Not assessed

This lab introduces the database environment we will use in CMPU 4003 and the Campus Events case study that will be used throughout CMPU4003. You have already studied relational databases, SQL, transactions and concurrency, indexes, query optimisation and NoSQL databases. The lab provides a short prerequisite check and introduces the broader architectural questions explored in Advanced Databases.

1. Setup your Database Environment for CMPU 4003

During CMPU4003 we will mainly work with two database systems:

- **PostgreSQL** – used for relational data, SQL, JSON, query performance, data warehousing and partitioning.
- **Redis** – used for key-value and in-memory data, Redis data structures, search, partitioning, replication and distributed database behaviour.

You have two options for running these databases:

- **Option A – Cloud environment:** Use **Supabase** for PostgreSQL and **Redis Cloud** for Redis. This is the recommended option for most students because there is little local configuration and you can access your databases from different computers.
- **Option B – Local Docker environment:** Use the supplied Docker Compose configuration to run PostgreSQL, pgAdmin, Redis and Redis Insight on your own computer. Choose this option if you prefer to work locally or already use Docker.

Which environment should I use?

For most labs you should be able to use the cloud-hosted PostgreSQL and Redis services. If you prefer to work locally, the supplied Docker Compose configuration can be used to run PostgreSQL, Redis, pgAdmin and Redis Insight locally. Both approaches provide the database functionality required for the early part of the module.

Unless you prefer to work with containers, choose **Option A – Supabase + Redis Cloud**.

Later in the module, when we investigate **replication, partitioning and distributed databases**, everyone will use a separate supplied multi-node Docker environment. You do **not** need to configure this yet.

Setup Instructions

Service	Used for	CMPU4003 role	Instructions
Supabase	PostgreSQL	Relational design, JSONB, SQL, warehousing, indexing, query plans and partitioning	Create a free Supabase account/project. Select West EU (Ireland) as the region. Use the SQL Editor to create and query the Campus Events schema (which you will create).
Redis Cloud	Redis	Key-value and in-memory data, Redis data structures, search, replication, partitioning and distributed-database concepts	Create a Redis account and a free 30 MB database . Do not select a paid Essentials or Pro plan. You may need to use an incognito/private browser window. Connect using Redis Insight.
Local Docker <i>(optional)</i>	PostgreSQL + pgAdmin + Redis + Redis Insight	Local alternative to the cloud environment, supporting the PostgreSQL and Redis work used in the early part of the module	Install and start Docker Desktop. Use the supplied Docker Compose file to start PostgreSQL, Redis, pgAdmin and Redis Insight. Choose this instead of Supabase + Redis Cloud if you prefer to work locally.
Multi-node Docker <i>(later in module)</i>	Multiple PostgreSQL and/or Redis database instances	Replication, clustering, partitioning and distributed-database experiments	No setup required yet. A separate Docker Compose environment will be supplied when required. You will use it to run multiple database nodes and investigate replication, distribution and failure behaviour.

2. The Campus Events System

TU Dublin is developing a Campus Events platform. Students can browse and register for events held at different venues. Events belong to categories and have a maximum capacity. The system records registrations and attendance.

```
STUDENT(student_id, name, email, programme)
VENUE(venue_id, name, building, capacity)
CATEGORY(category_id, category_name)
EVENT(event_id, title, event_date, venue_id, category_id, capacity)
REGISTRATION(registration_id, student_id, event_id, registered_at, status, attended)
```

Review the relational design

1. Identify the primary and foreign keys.
2. What is the relationship between STUDENT and EVENT?
3. Why is REGISTRATION required?
4. How could the database prevent the same student registering twice for the same event?

3. Setup Campus Events

If you have an installation of PostgreSQL from last year you can reuse that and just complete the setup of Redis. Otherwise choose Option A or Option B as you prefer.

Option A – Cloud Environment

PostgreSQL - Supabase

1. Create/sign in to your Supabase account and create a project for CMPU4003.
When creating a project – make sure you change **Region to West EU (Ireland)**
2. Open the SQL Editor
3. Run the supplied `campus-events.sql` script.
4. Confirm that the STUDENT, VENUE, CATEGORY, EVENT and REGISTRATION tables have been created.

Verify the data:

```
SELECT COUNT(*) FROM student;
```

```
SELECT COUNT(*) FROM event;
```

```
SELECT COUNT(*) FROM registration;
```

Redis - Redis Cloud

1. Create/sign in to your Redis account and create a free Redis Cloud database.
You may have to do this from an incognito window
Click **Not ready? Try 30 MB for free” to create the free learning database.**
Do not select a paid Essentials or Pro plan.
2. Once the database has been created, click **Connect** and confirm that you can access it.

Option B – Local Docker Environment

A Docker Compose file is supplied with this lab. It creates four containers:

- **PostgreSQL** — relational database
- **pgAdmin** — browser-based PostgreSQL administration/query tool
- **Redis** — Redis database
- **Redis Insight** — browser-based Redis administration/query tool

You must have **Docker Desktop** installed and running.

Start the environment

Place the supplied compose.yaml file in a folder for CMPU4003. Open a terminal in that folder and run:

```
docker compose up -d
```

Check that all four containers are running: `docker compose ps`

You should see:

- cmpu4003-postgres
- cmpu4003-pgadmin
- cmpu4003-redis
- cmpu4003-redisinsight

Service	Access
PostgreSQL	localhost:5432
pgAdmin	localhost:5050
Redis	localhost:6379
Redis Insight	localhost:5540

Connect pgAdmin to PostgreSQL

- You can use another DBClient if you prefer.
- Open pgAdmin in your browser using localhost:5050.
- Sign in using the credentials specified in the supplied Compose file and register a new PostgreSQL server using:
 - Name: Campus Events
 - Host: postgres
 - Port: 5432
 - Maintenance Database: postgres
 - Username: postgres
 - Password: postgres
- **Note:** When connecting from pgAdmin to PostgreSQL, use postgres rather than localhost as the host. Both applications are running inside the same Docker network.
- Open the **pgAdmin Query Tool** and run the supplied `campus-events.sql` script.

Connect Redis Insight to Redis

- Open Redis Insight in your browser using localhost:5540.
- On the **Redis Databases** screen, select: **+ Connect existing database**
- Enter:
 - Host: redis
 - Port: 6379
 - Username: leave blank
 - Password: leave blank
 - TLS: off
- Give the connection a name such as **Campus Events Redis** and connect.
- **Note:** Use redis rather than localhost because Redis Insight and Redis are running as separate containers on the same Docker network.

From this point onwards

Once your PostgreSQL and Redis databases are available, the remainder of the lab is the same regardless of whether you are using the **cloud** or **Docker** environment.

- For PostgreSQL tasks, use the **Supabase SQL Editor** or **pgAdmin**, depending on your chosen environment.
- For Redis tasks, use Redis Insight/Workbench.

The exercises focus on the **database concepts rather than the hosting environment**.

4. PostgreSQL prerequisite check

Write SQL to answer the following. These are revision questions; you do NOT need to submit your answers.

1. **Display future events in date order.**

Hint: consider WHERE, ORDER BY and CURRENT_DATE Use a WHERE condition to compare the event date with the current date.

2. **Display event title, date, category and venue name.**

Hint: The information you need is stored across **three tables**.

Pick your starting table then look for foreign keys.

You will need more than one JOIN.

3. **Display each event and the number of active registrations, including events with no registrations.**

Hint: You need information from two tables.

Think carefully about the type of JOIN. Will an ordinary INNER JOIN help find events that have no registrations?

You will need to use COUNT() and GROUP BY.

Only registrations with the appropriate status should be counted.

Be careful where you put the condition on status — could putting it in the WHERE clause cause events with no registrations to disappear?

4. **Challenge: Display only events with more than five active registrations.**

Start with your solution to Question 3.

You need to filter the results after the registrations have been counted.

Which SQL clause allows you to filter groups after GROUP BY?

5. Concurrency and performance

Concurrency

An event has a capacity of **100** and currently has **99 active registrations**. When a student registers, the application:

1. Counts the current registrations.
2. Checks whether the count is less than the event capacity.
3. If a place is available, inserts the new registration.

Now suppose **Student A and Student B attempt to register at almost exactly the same time**. Consider what could happen if both requests count the registrations **before either new registration has been inserted**.

- What registration count could Student A see?
- What registration count could Student B see?
- Could both requests therefore decide that a place is available?
- What could the final number of registrations become?
- What **concurrency problem** is occurring?
- What database mechanisms from your previous study could be used to prevent this?

Performance Check

Now run the following:

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT e.title,
       COUNT(r.registration_id) AS registrations
FROM event e
LEFT JOIN registration r
      ON e.event_id = r.event_id
GROUP BY e.event_id, e.title;
```

Identify a **scan**, a **join**, **estimated and actual rows**, **buffer usage**, and **execution time**.

Then consider:

Why might PostgreSQL choose a different plan if REGISTRATION contained 10 million rows?

You do not need to optimise the query yet. Consider what additional information you would need before deciding whether optimisation is necessary and what approach you would take.

6. Redis: first contact

Redis is different from the relational databases you have worked with previously. PostgreSQL organises data primarily into **tables, rows and columns** and is queried using **SQL**. Redis is primarily a **key-value database**: data is stored and retrieved using a unique **key**.

Redis does not use SQL. Instead, you interact with it using **Redis commands**. A command normally consists of the operation followed by the key and, where required, a value.

For example: `SET event:1:title "AI Careers Evening"` stores the value "AI Careers Evening" using the key `event:1:title`.

Using your Redis database, open **Redis Insight → Workbench**.

- If using Redis Cloud, select **Connect → Redis Insight (browser) → Workbench**.
- If using Docker, open Redis Insight at `localhost:5540`, select **Campus Events Redis**, and open **Workbench**.

Try the following commands:

```
PING
```

Checks that the Redis server is responding. A working server should return PONG.

```
SET event:1:title "AI Careers Evening"
```

SET stores a value against a key. Here, `event:1:title` is the key and "AI Careers Evening" is its value.

```
GET event:1:title
```

GET retrieves the value associated with the specified key.

```
INCR event:1:views
```

```
INCR event:1:views
```

INCR treats the value as an integer and increments it by one. If the key does not already exist, Redis starts it at 0 before incrementing it.

```
GET event:1:views
```

The event title now exists in PostgreSQL and Redis.

This should now return 2.

The naming convention used here is a common way of constructing meaningful Redis keys:

```
event:1:title
```

```
event:1:views
```

The `:` characters help organise the key name, but unlike a relational database, these are **not tables or columns**.

At this stage, the objective is simply to become familiar with the Redis **key-value model and command-based interface**. Later we will look at Redis data structures, document-oriented data and when Redis may be more appropriate than—or used alongside—a relational database such as PostgreSQL.

Consider:

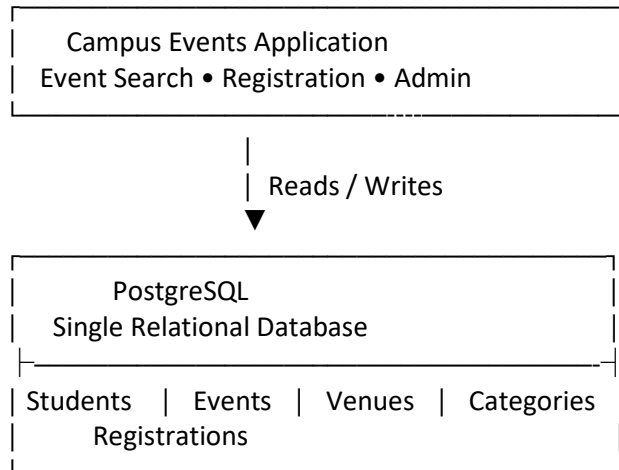
Why might an application deliberately store or derive the same information in two database systems e.g. PostgreSQL and Redis?

You don't need a complete answer yet.

7. From Database Systems to Advanced Databases

Assume Campus Events has now been running for several years. It has 100,000 students, 50,000 events, 10 million registrations, several years of history, heavy mobile read traffic, increasingly varied event data, substantial management analytics requirements, and a requirement to remain available following server failure.

Initial architecture: The application uses a single relational PostgreSQL database for all operational data and workloads.



Architecture challenge

1. What problems might emerge as the system grows?
2. Would you continue to represent all event data in exactly the same relational form?
3. Would all data necessarily need to remain in a relational database? Could a different data model or database architecture be more appropriate for some of the data or workloads?
4. Should operational registration workloads and historical analytical workloads use the same database design?
5. What architectural changes might be required when one database server is no longer sufficient?
6. What information about the workload would you want before making these decisions?

8. Final question

If PostgreSQL can already store and query all of the Campus Events data, why might we need other database architectures or data models at all?

For example, what might make us consider:

- a **document-oriented database** for flexible or evolving event data;
- a **data warehouse** for large-scale historical and analytical queries;
- a **distributed database architecture** when data and workloads need to operate across multiple nodes;
- **replication** when availability and resilience become important;
- **partitioning** when the volume of data or workload becomes too large for a single database instance;
- a **vector database or vector search capability** when we want to search for events based on semantic similarity rather than exact values or keywords?

You do not need to answer these questions completely yet.

During CMPU4003, we will investigate **when these approaches are appropriate, what problems they solve, what trade-offs they introduce, and how they can be combined with relational database systems.**

Rather than simply asking whether a database can be created and queried, we will ask how data should be modelled and physically organised; how workloads affect design; when different data models or technologies are appropriate; how operational and analytical data should be organised and integrated; how data should be replicated and partitioned; what happens when servers or networks fail; and how competing database technologies can be compared.

